Seaborn Visualization: Basics to Advanced

Agenda

- Introduction to Seaborn
- Plotting Graphs Using Seaborn
- Violin Plot
- Pair Plot
- Heatmap
- Joint Plot
- Swarm Plot
- Plotting 3D Graphs for Multiple Columns Using Seaborn
- Introduction to Plotly
- Plotting a Graph Using Plotly
- Scatter Plot
- Box Plot
- Scatter Matrix
- Heatmap

1. Introduction to Seaborn

Seaborn is a Python library for statistical data visualization that builds on Matplotlib.

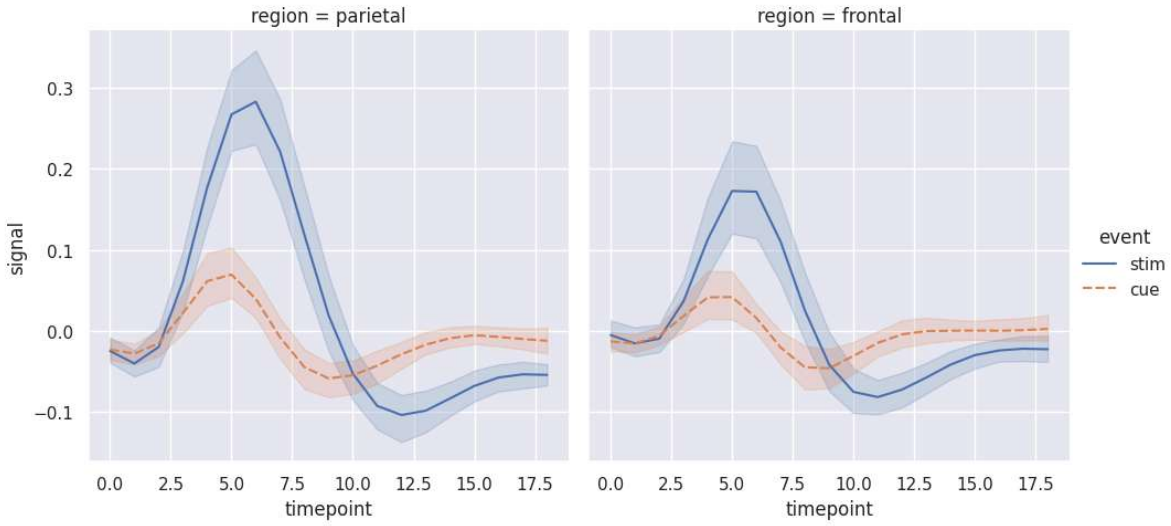
- It provides an interface for creating attractive and informative statistical graphics.
- It comes with several built-in themes and color palettes to make creating aesthetically pleasing visualizations easy.
- It is particularly well-suited for exploring complex datasets with multiple variables.

The code below leverages Seaborn's functionality to create a specialized line plot for visualizing fMRI data with distinct regions and events.

- Seaborn simplifies the process of creating complex visualizations and provides additional features for customization and exploration of data patterns.

```
import seaborn as sns
sns.set_theme()
fmri = sns.load_dataset("fmri")
sns.relplot(
    data=fmri, kind="line",
    x="timepoint", y="signal", col="region",
    hue="event", style="event",
)
```

```
<seaborn.axisgrid.FacetGrid at 0x7f0a3bff7ee0>
```

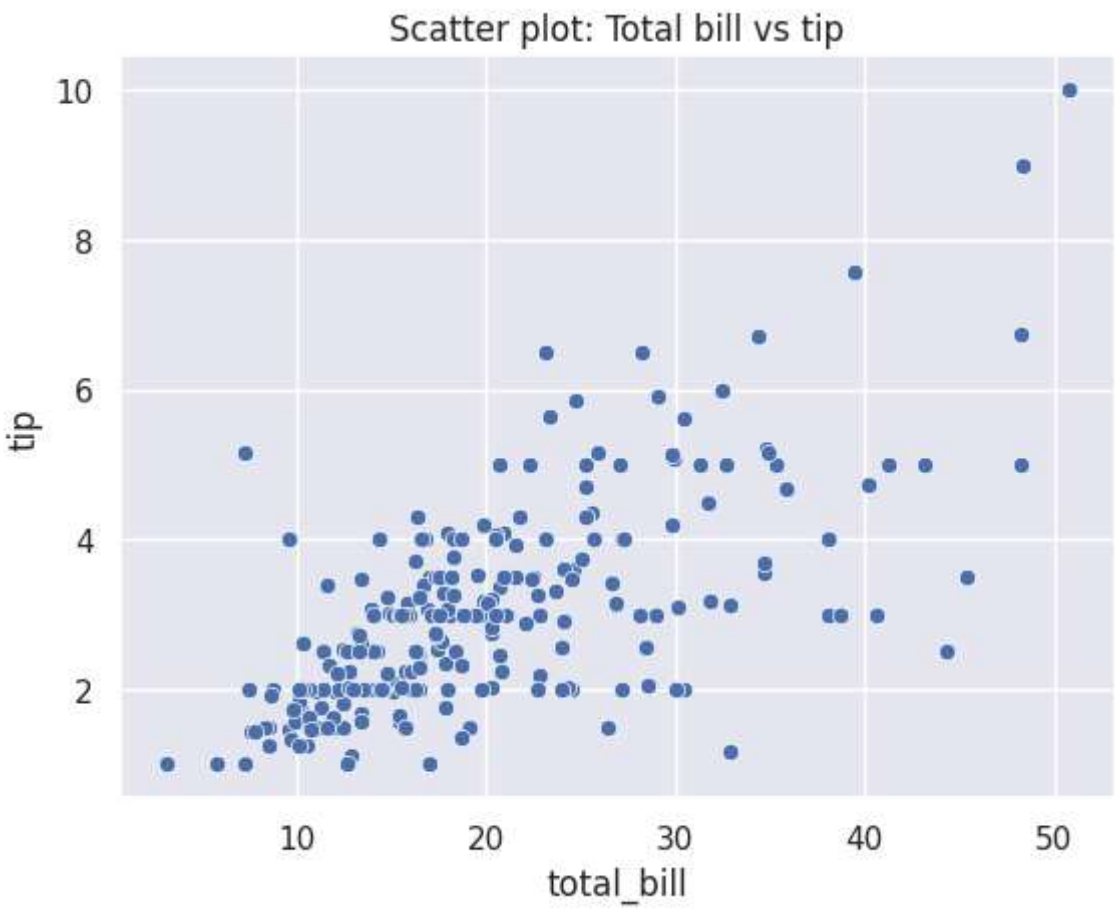


2. Plotting Graphs Using Seaborn

Note: Previously, you explored these plot types with Matplotlib. Now, let's demonstrate how Seaborn achieves similar visualizations with enhanced styling and simplicity.

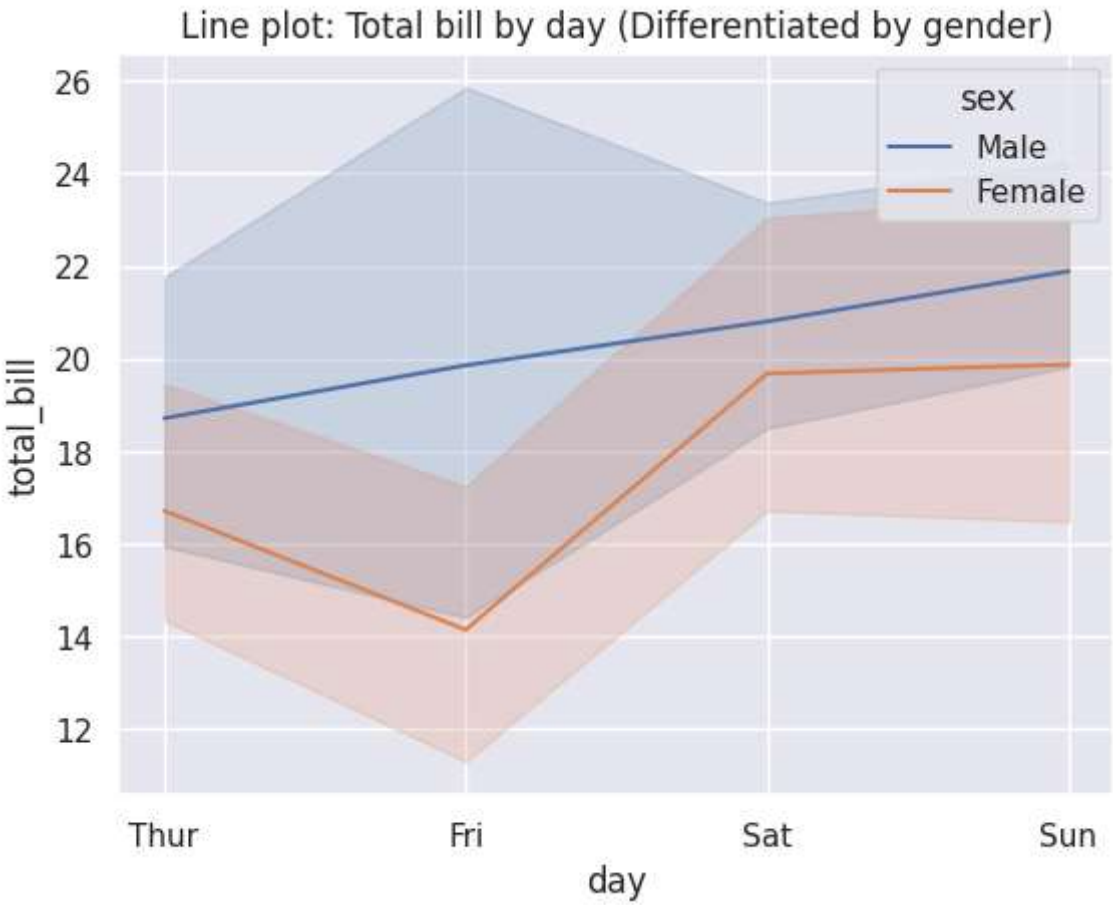
```
import seaborn as sns
import matplotlib.pyplot as plt

# Load a sample dataset
tips = sns.load_dataset('tips')
```

```
# Scatter Plot
sns.scatterplot(x='total_bill', y='tip', data=tips)
plt.title('Scatter plot: Total bill vs tip')
plt.show()
```



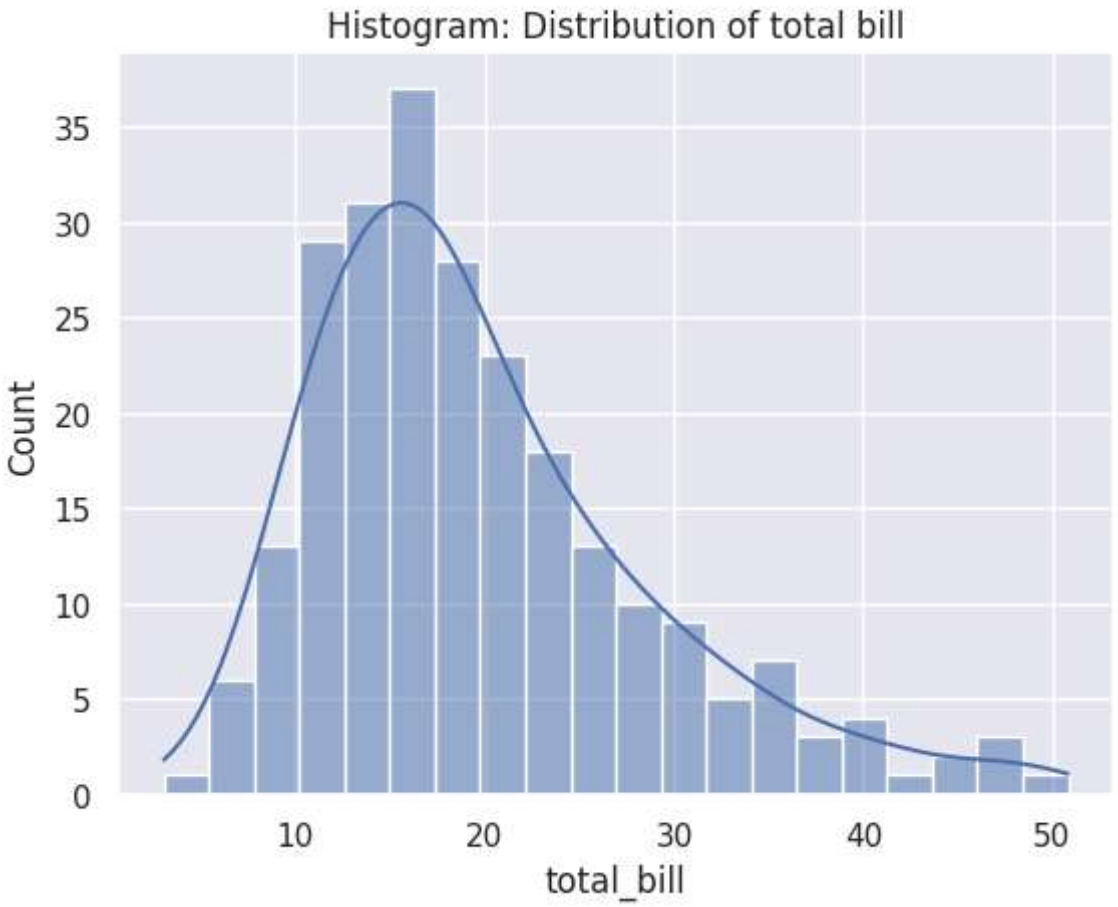
- Based on the scatter plot, you can infer a positive relationship between the total bill and tip amount, indicating that customers tend to tip more when they spend more on their meals.
- There is still variability in tip amounts for a given total bill, suggesting that factors other than the bill amount may also influence the tips customers leave.

```
# Line Plot
sns.lineplot(x='day', y='total_bill', data=tips, hue='sex')
plt.title('Line plot: Total bill by day (Differentiated by gender)')
plt.show()
```



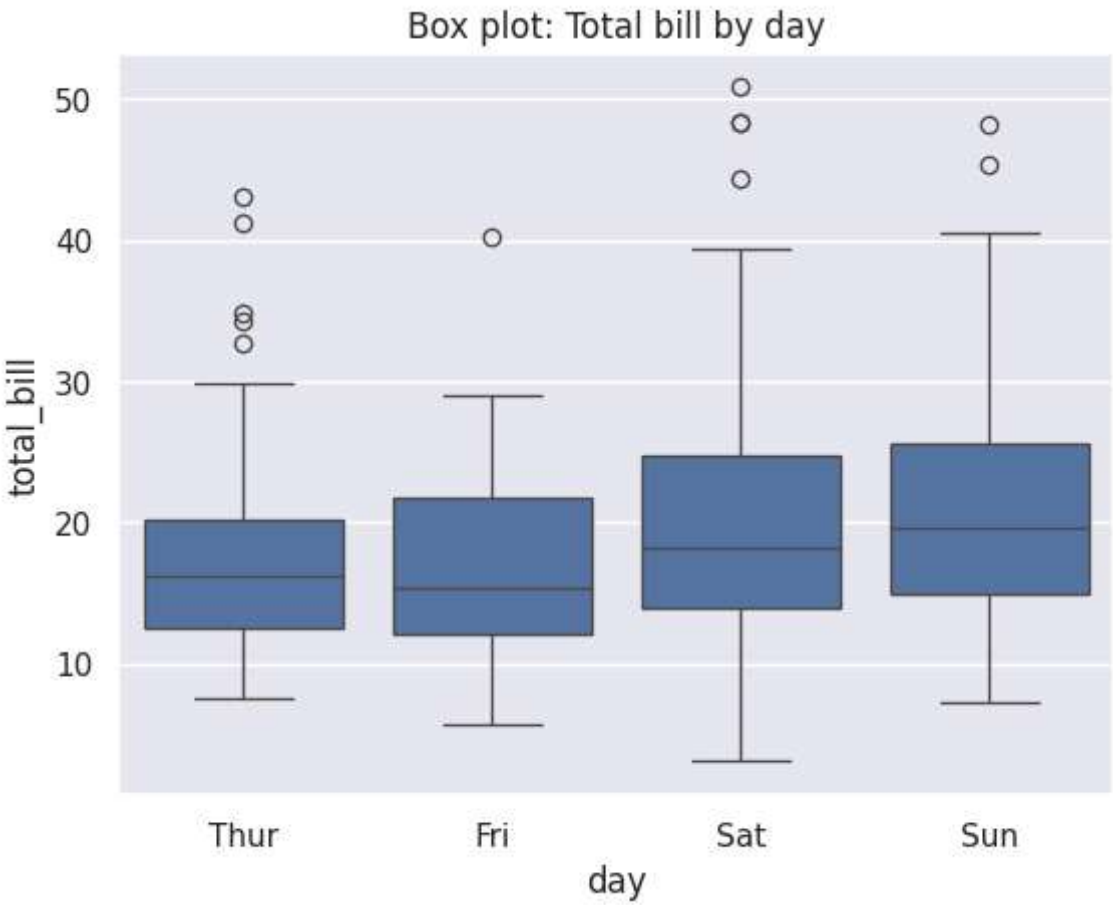
- The lines on the plot represent the trend of total bill amounts across different days of the week, segmented by gender (male and female).
- Based on the line plot, you can infer that the total bill paid by males is considerably higher than that paid by females.

```
# Histogram
sns.histplot(tips['total_bill'], bins=20, kde=True)
plt.title('Histogram: Distribution of total bill')
plt.show()
```



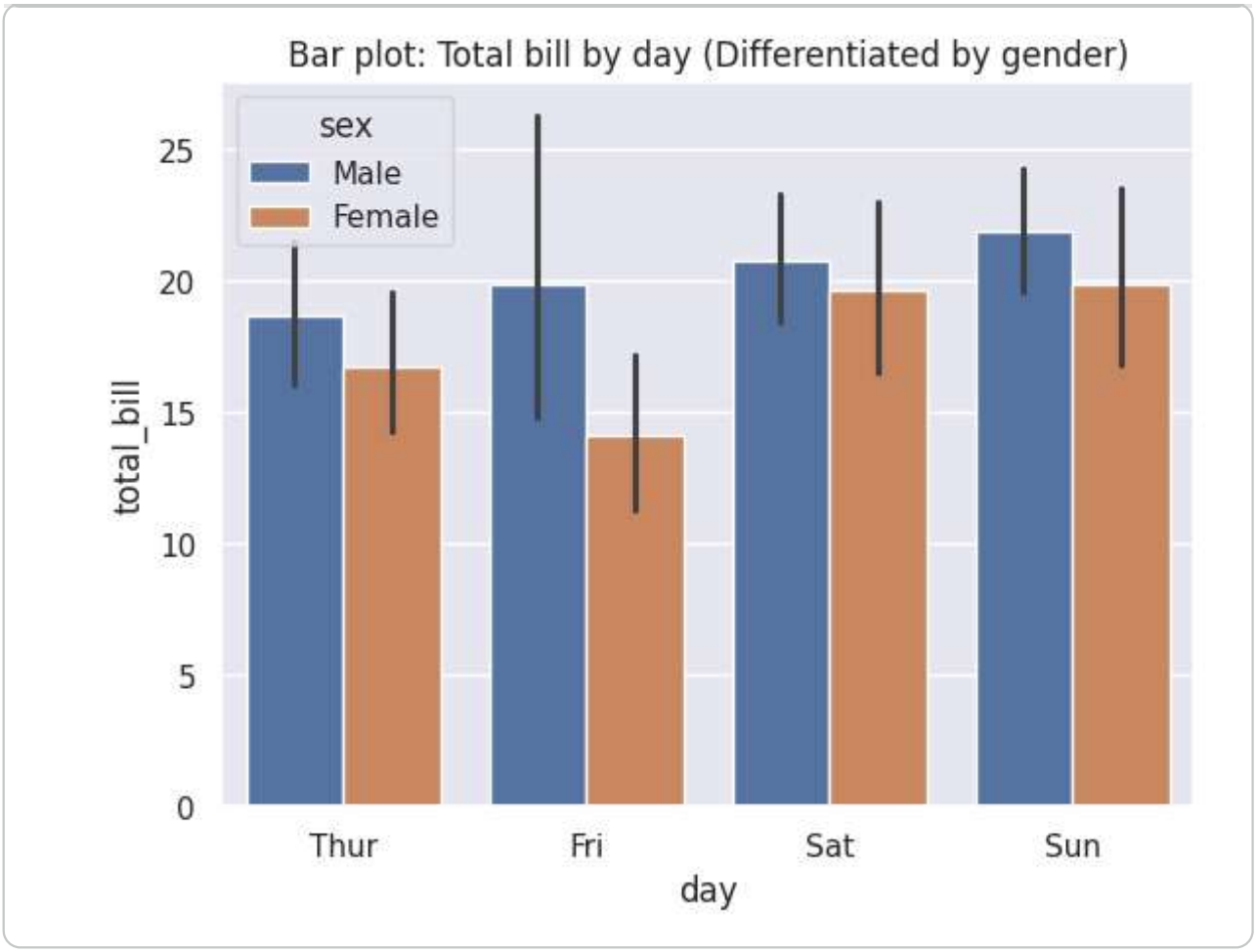
- The histogram displays the distribution of total bill amounts across the dataset.
- Each bar on the plot represents a range or bin of total bill amounts, and the height of the bar corresponds to the frequency of occurrences within that range.
- The data is right-skewed, which means the mode, median, and mean are arranged, respectively.
- Based on the chart, you can infer that most of the total bills lie in the 10 to 20 range.

```
# Box Plot
sns.boxplot(x='day', y='total_bill', data=tips)
plt.title('Box plot: Total bill by day')
plt.show()
```



- By examining the box plot, you can identify the central tendency and spread of total bill amounts for each day of the week.
- The length of the box provides insights into the variability of total bill amounts, with longer boxes indicating greater variability.
- The position of the median line within each box helps you compare the typical bill amounts across different days.
- You can see that the median on Saturday and Sunday is significantly higher than on Thursday and Friday.
- The appearance of data points beyond the whiskers indicates the presence of outliers within the dataset.

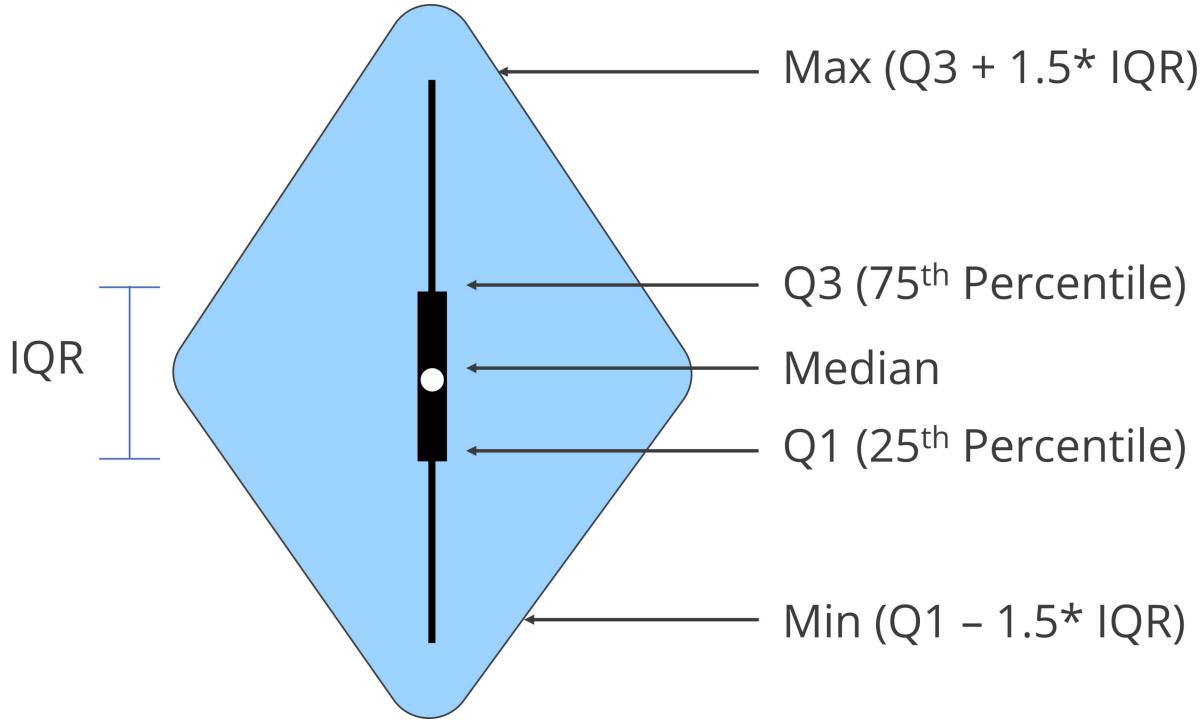
```
# Bar Plot
sns.barplot(x='day', y='total_bill', data=tips, hue='sex')
plt.title('Bar plot: Total bill by day (Differentiated by gender)')
plt.show()
```



- The bar plot facilitates a comparison of average total bill amounts across different days of the week while also considering the gender factor.
- Based on the chart, you can see that there is a disparity in spending patterns between males and females.
- This disparity is even more pronounced on Thursday and Friday as compared to Saturday and Sunday.
- Error bars provide a visual indication of the spread of data points around the mean value for each category in the plot.

3. Violin Plot

A violin plot merges the features of kernel density plots and box plots to show a numerical variable's distribution across categories.

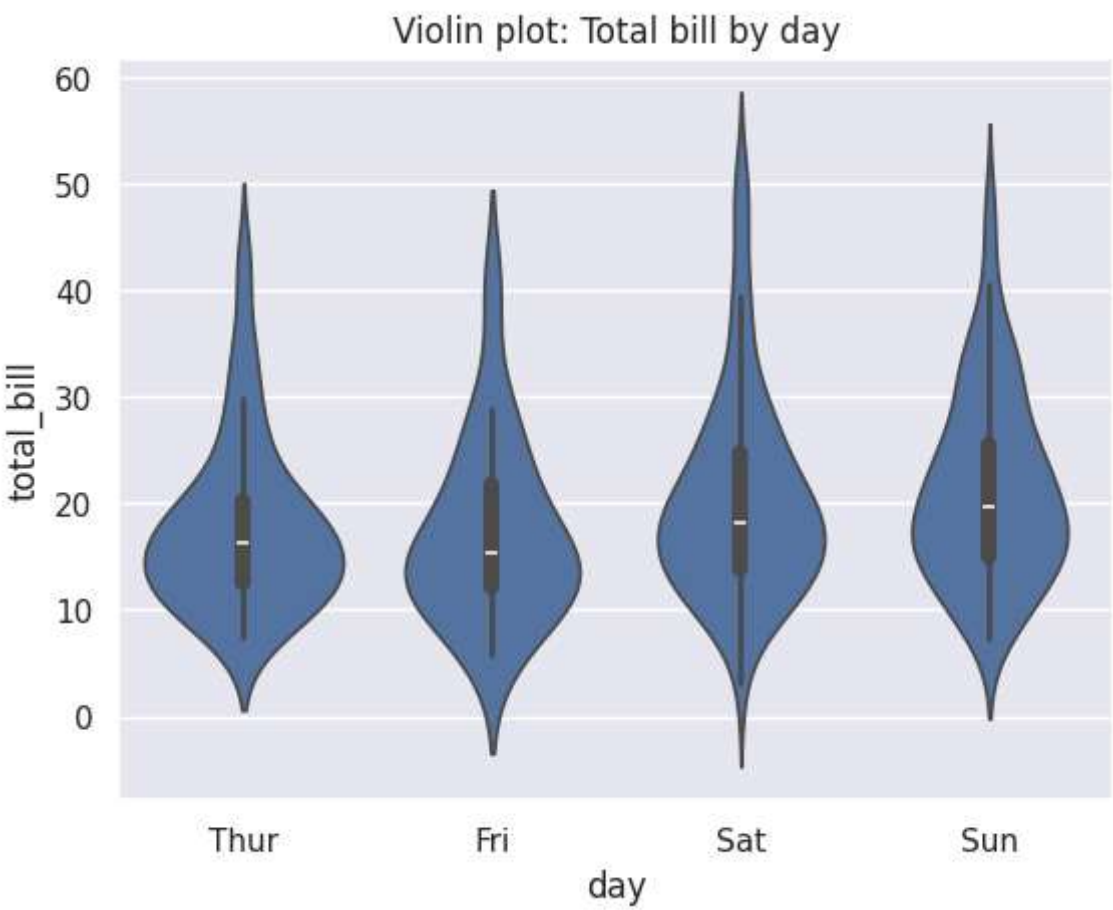


In a violin plot:

- The thick black line in the center represents the median.
- The white dot inside the violin represents the mean.
- The width of the violin at any given point represents the density of the data at that value. Wider sections indicate a higher density of data points.
- The lines extending from the violin, known as whiskers, represent the rest of the distribution outside the interquartile range.
- Any points beyond the whiskers are considered outliers and are plotted individually.

This plot helps you visualize the distribution of total bills for each day, highlighting data point densities and key statistical measures such as the median and interquartile range.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load a sample dataset
tips = sns.load_dataset('tips')

# Violin Plot
sns.violinplot(x='day', y='total_bill', data=tips)
plt.title('Violin plot: Total bill by day')
plt.show()
```



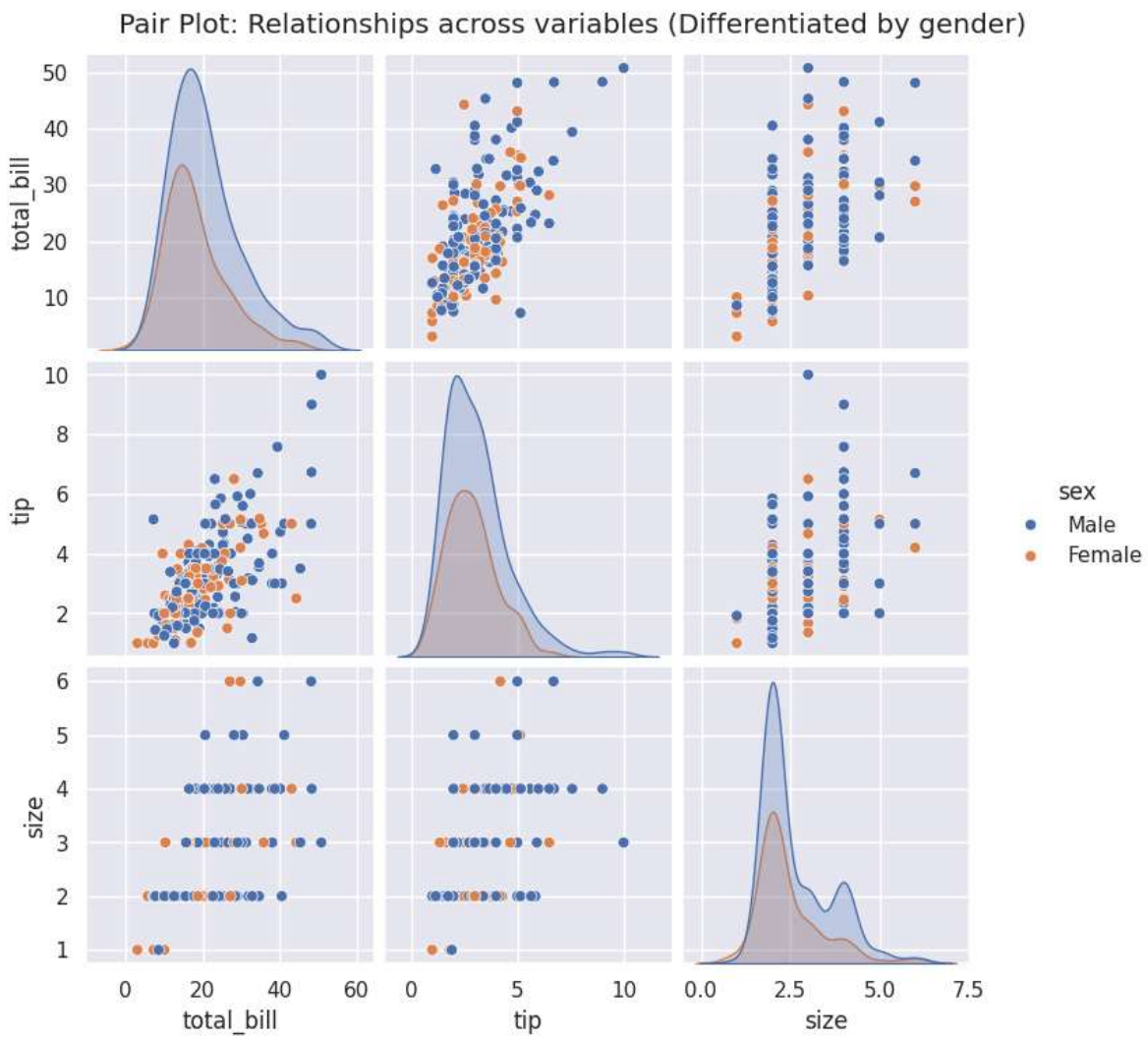
4. Pair Plot:

A pair plot illustrates the relationships between numerical variables in a dataset using scatter plots, distributions, and color-coded categories.

- It is useful for understanding how different numerical variables relate to each other and how these relationships vary based on a given variable.

This pair plot will help you visualize the relationship across all the variables with respect to the sex column.

```
# Pair Plot
sns.pairplot(tips, hue='sex')
plt.suptitle('Pair Plot: Relationships across variables (Differentiated by g
plt.show()
```



- The above plot shows spending behavior by gender.
- The total bill for males is higher than for females.
- Males have paid more tips than females.

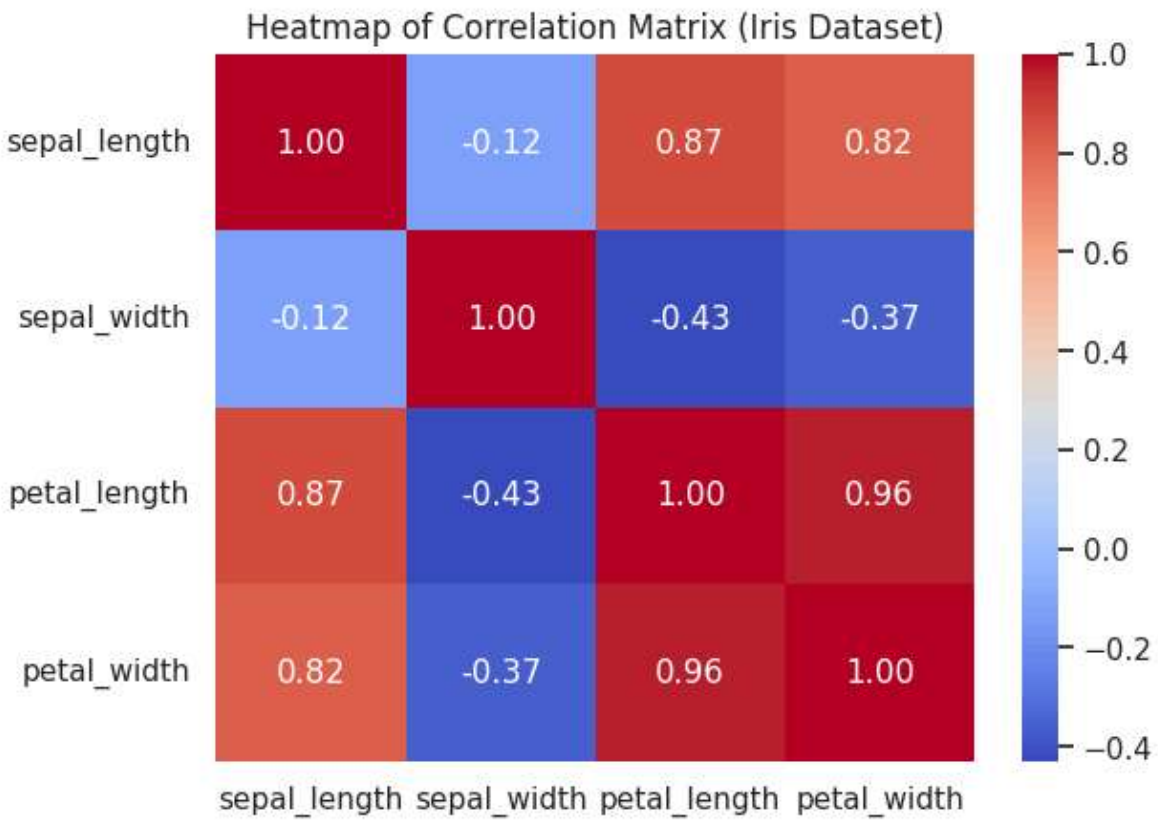
5. Heatmap:

A heatmap visualizes the correlation matrix of numerical variables in a dataset, using color gradations to represent the strength and direction of correlations.

- It allows for the quick identification of relationships between variables, with warmer colors indicating stronger correlations and cooler colors indicating weaker or negative correlations.

This map shows the correlation values of different pairs of variables.

```
# Load a sample dataset
iris = sns.load_dataset('iris')

# Calculate correlation matrix for numeric columns only
correlation_matrix = iris.select_dtypes(include='number').corr()

# Plot heatmap
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Heatmap of Correlation Matrix (Iris Dataset)')
plt.show()
```



- The pairs with high correlation values are shown in warmer colors.
- The pairs with low correlation values are shown in cooler colors.

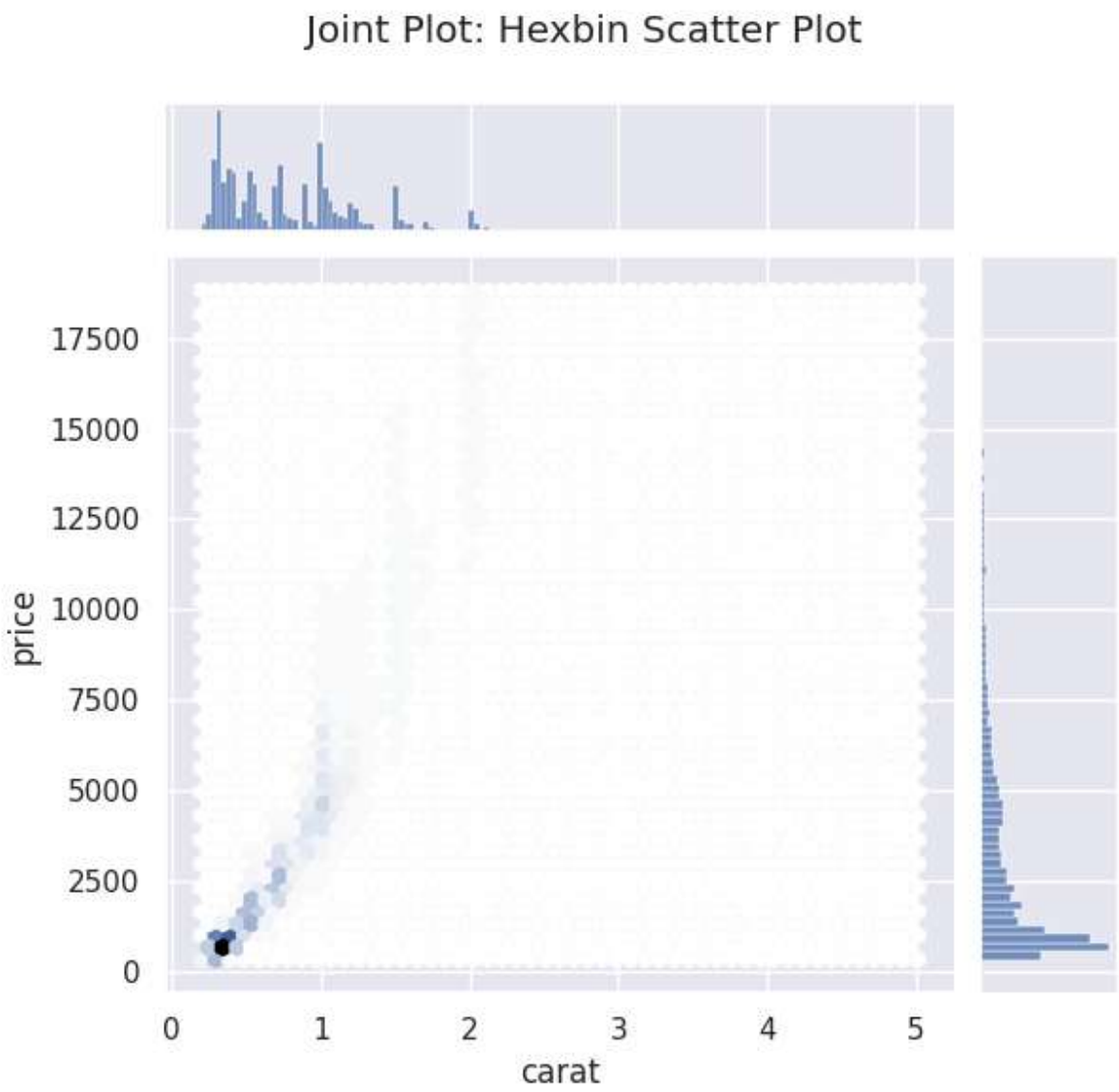
6. Joint Plot

A joint plot reveals the bivariate distribution of two numerical variables using hexagonal bins to represent point density.

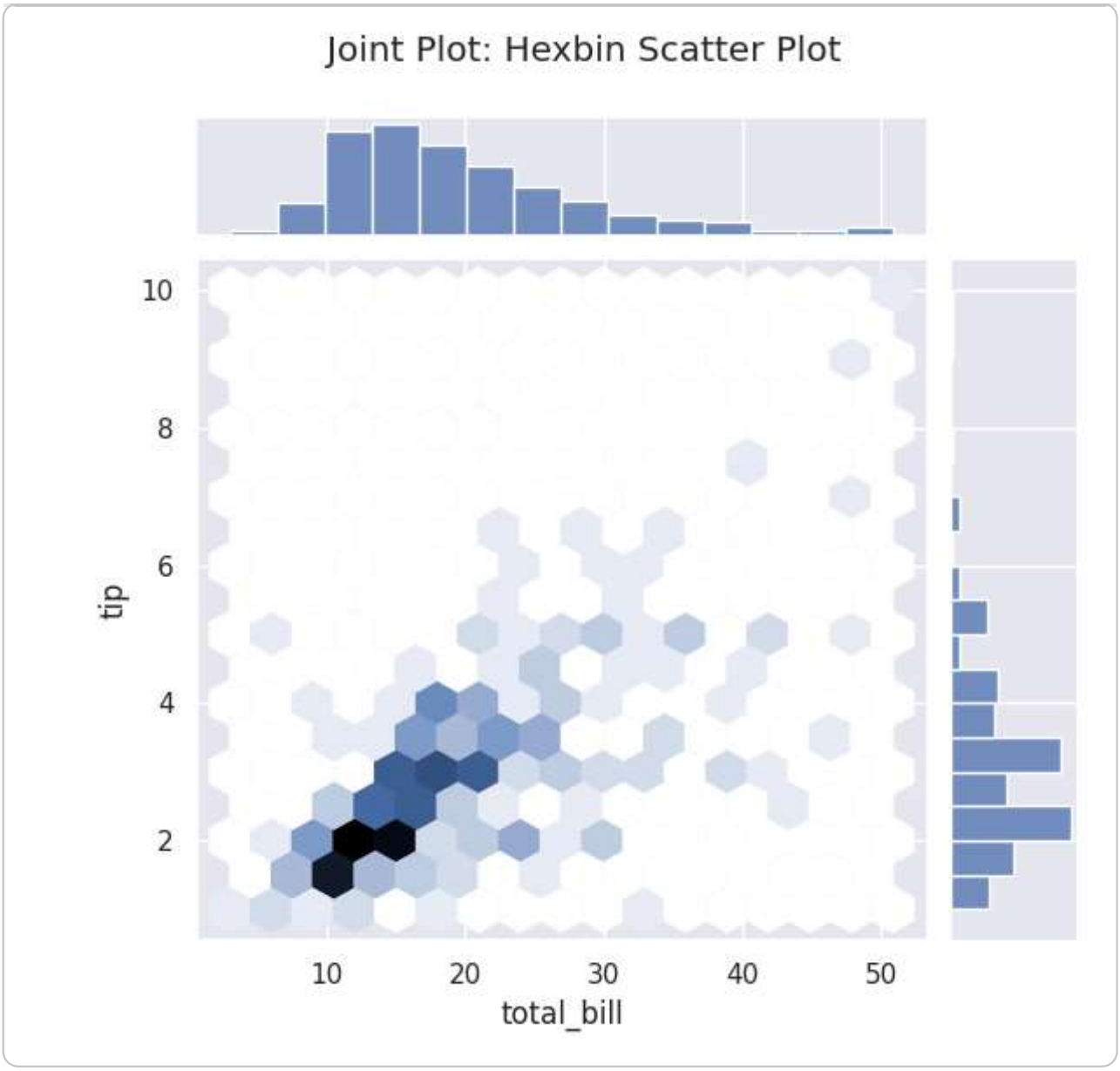
- It is a type of plot in data visualization that combines multiple plots to show the relationship between two variables in a dataset.
- It typically includes a scatterplot to visualize the individual data points and additional information such as histograms, kernel density estimates, or regression lines to provide more context about the distribution.

This joint plot helps to identify patterns and concentrations in the relationship between **total_bill** and **tip**, with darker hexagons indicating a higher point density.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the diamonds dataset
diamonds = sns.load_dataset('diamonds')

# Create a Joint Plot
joint = sns.jointplot(x='carat', y='price', data=diamonds, kind='hex')

# Adjust the title position
plt.subplots_adjust(top=0.9)

# Set the title for the figure
joint.fig.suptitle('Joint Plot: Hexbin Scatter Plot')

plt.show()
```




```
# Create a Joint Plot
joint = sns.jointplot(x='total_bill', y='tip', data=tips, kind='hex')
# Adjust the title position
plt.subplots_adjust(top=0.9)  # Adjust the top space to make room for the ti
# Set the title for the figure
joint.fig.suptitle('Joint Plot: Hexbin Scatter Plot')
plt.show()
```



- The joint plot shows the relationship between **total_bill** and **tip** using hexagonal bins to represent the data density.
- Based on the plot, you can infer that for the total bill, the density of the data is high in the 10 to 20 range.
- You can also infer that for the tips, the density of the data is high in the 1 to 3 range.

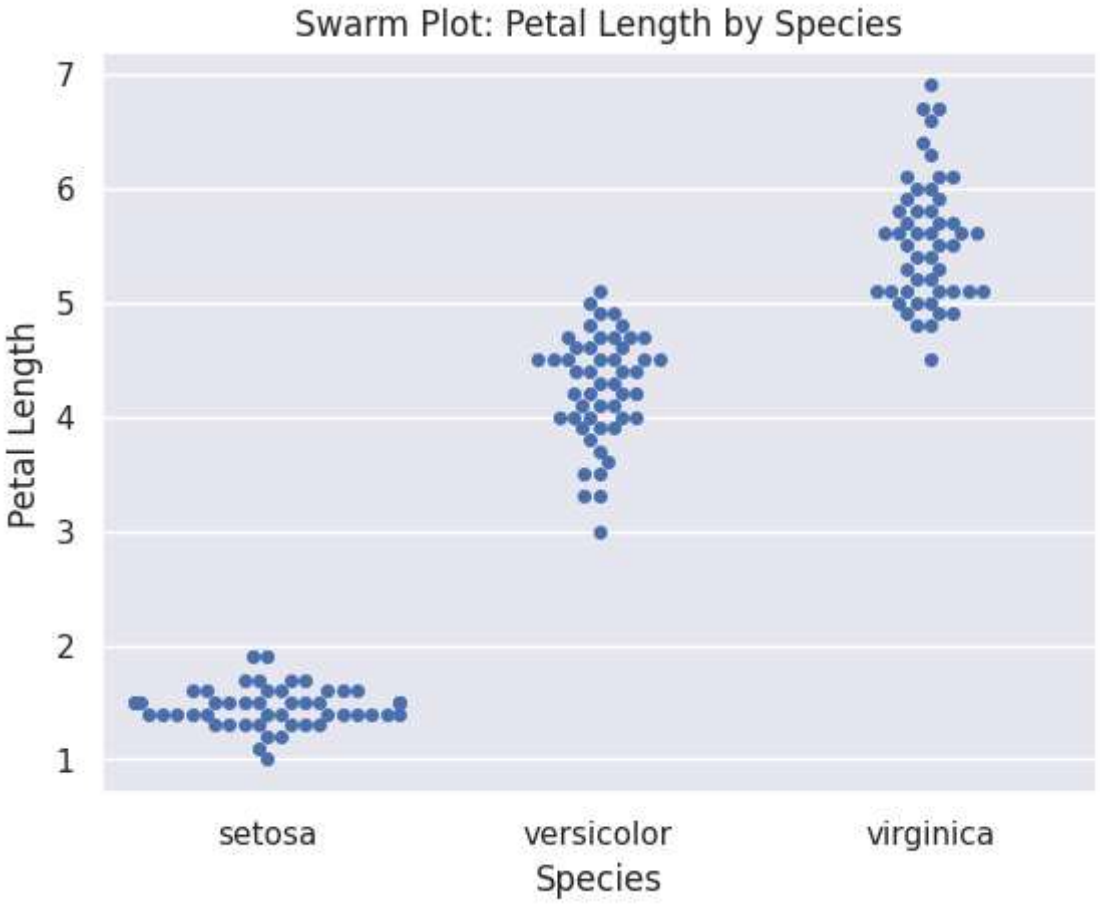
7. Swarm Plot

A swarm plot is a categorical scatter plot that arranges individual data points without overlapping.

- It is useful for visualizing the distribution of variables across different days, with each point representing an individual entry.
- It helps identify patterns and concentrations without losing granularity due to point overlap.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load the iris dataset
iris = sns.load_dataset('iris')

# Create a Swarm Plot
sns.swarmplot(x='species', y='petal_length', data=iris)

# Set the title and labels
plt.title('Swarm Plot: Petal Length by Species')
plt.xlabel('Species')
plt.ylabel('Petal Length')

plt.show()
```

```
/usr/local/lib/python3.10/dist-packages/seaborn/categorical.py:3398: UserWarn
  warnings.warn(msg, UserWarning)
```

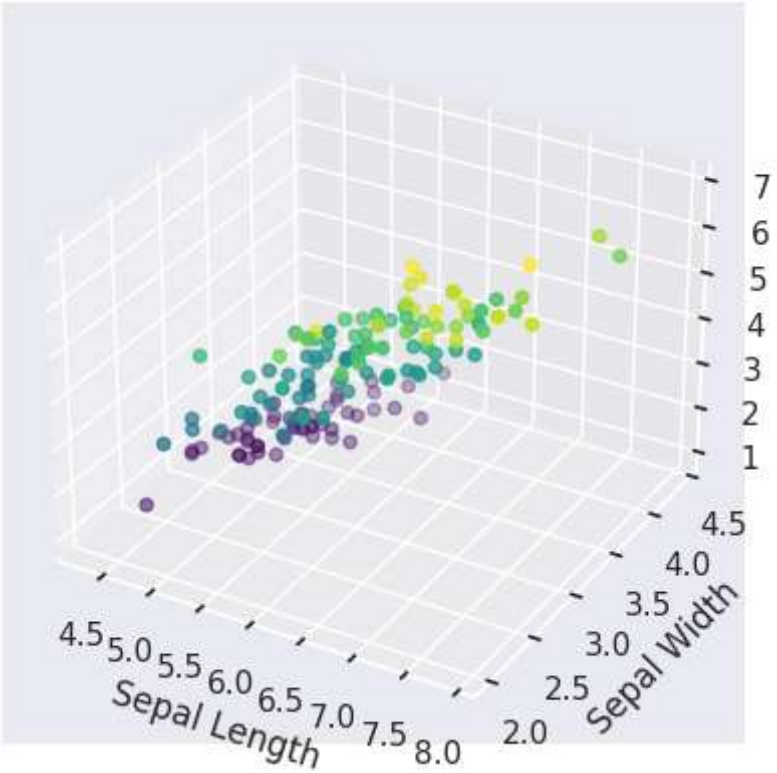


- The swarm plot allows you to visually compare the distribution of petal lengths across different species of iris flowers: Setosa, Versicolor, and Virginica.
- You can observe the range of petal lengths within each species category by examining the spread of points along the y-axis.
- The spread of petal lengths for Setosa falls within the range of 1 to 2. For Versicolor, it falls within the range of 3 to 5. For Virginica, it ranges from 4.5 to 7.

8. Plotting 3D Graphs for Multiple Columns Using Seaborn

- Seaborn's 3D graphs provide insights into relationships among multiple variables, aiding in the discovery of intricate patterns and outliers.

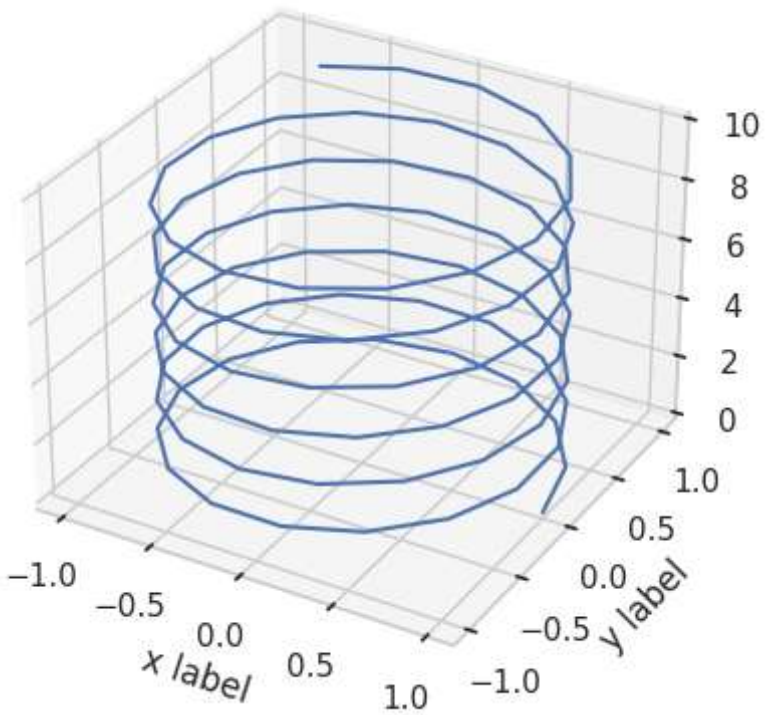
```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns

# Load an example dataset
iris = sns.load_dataset('iris')

# Create a 3D scatter plot
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(iris['sepal_length'], iris['sepal_width'], iris['petal_length'],

# Set labels
ax.set_xlabel('Sepal Length')
ax.set_ylabel('Sepal Width')
ax.set_zlabel('Petal Length')

# Show the plot
plt.show()
```



- 3D graphs provide additional depth perception, which may aid in understanding complex relationships in three-dimensional space.
- 3D plots can visualize relationships between three variables simultaneously, which might be challenging to represent accurately in a 2D plot.
- Based on this plot, you can see the distribution of data based on sepal length, sepal width, and petal length.

Plot a 3D Spiral Graph

```
# Import necessary libraries
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Generate 3D data
z1 = np.linspace(0, 10, 100)
x1 = np.cos(4 * z1)
y1 = np.sin(4 * z1)

# Set Seaborn style to whitegrid
sns.set_style('whitegrid')

# Create a 3D axes
ax = plt.axes(projection='3d')

# Plot the 3D curve
ax.plot3D(x1, y1, z1)

#set the labels
ax.set_xlabel('x label')
ax.set_ylabel('y label')

# Display the 3D plot
plt.show()
```

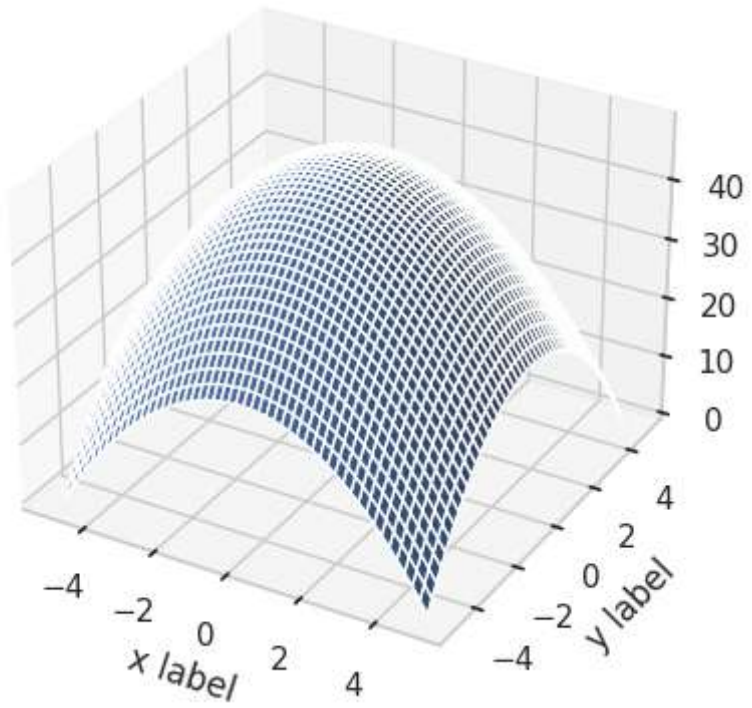


- The resulting chart is a 3D plot of a helix that looks like a spring.
- The x and y coordinates trace a circular path in the x-y plane, while the z coordinate increases linearly.

Create 3D Surface Using Seaborn

```
# Define a function to compute z values based on x and y
def return_z(x, y):
    return 50 - (x**2 + y**2)

# Set Seaborn style to whitegrid
sns.set_style('whitegrid')

# Generate 2D grid of x and y values
x1, y1 = np.linspace(-5, 5, 50), np.linspace(-5, 5, 50)
x1, y1 = np.meshgrid(x1, y1)

# Compute z values using the defined function
z1 = return_z(x1, y1)

# Create a 3D axes
ax = plt.axes(projection='3d')

# Plot the 3D surface
ax.plot_surface(x1, y1, z1)

#set the labels
ax.set_xlabel('x label')
ax.set_ylabel('y label')
```

```
# Display the 3D plot
plt.show()
```



- The resulting chart is a 3D surface plot of a paraboloid shape.
- The surface is curved upwards, with the highest point at the center (x=0, y=0) and the height (z value) decreasing as the distance from the center increases.

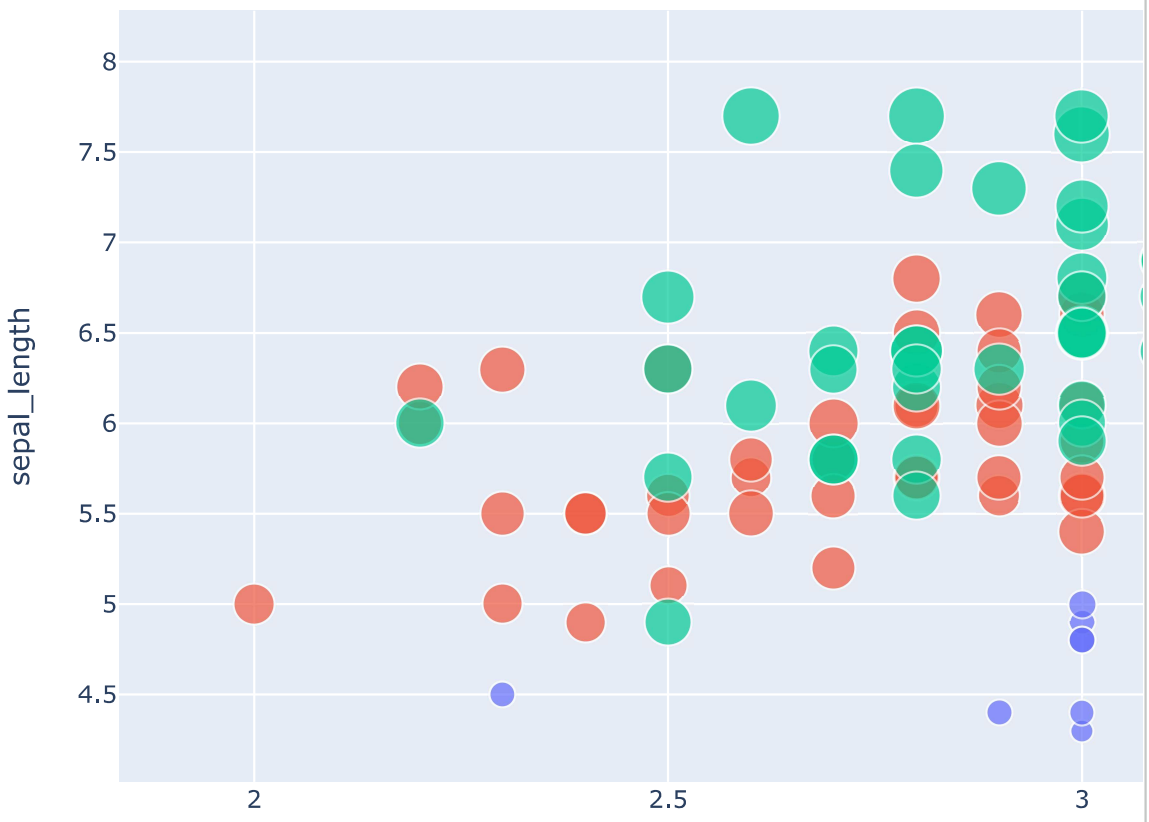
Introduction to Plotly

Plotly, a Python library for data visualization, supports creating interactive, web-based visualizations.

- It allows users to create a wide range of charts, graphs, and dashboards for exploratory data analysis and presentation purposes.
- It supports both static and dynamic visualizations and is particularly well-suited for creating interactive plots that can be embedded in web applications and notebooks.
- Plotly provides interactive visualizations. This means that you can zoom, pan, hover over data points to see additional information, and toggle specific data series on and off directly in the plot.

Now, let's visualize various plots using Plotly.

```
import plotly.express as px

# Create a sample DataFrame
df = px.data.iris()

# Create a scatter plot
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species", siz

# Show the interactive plot
fig.show()
```

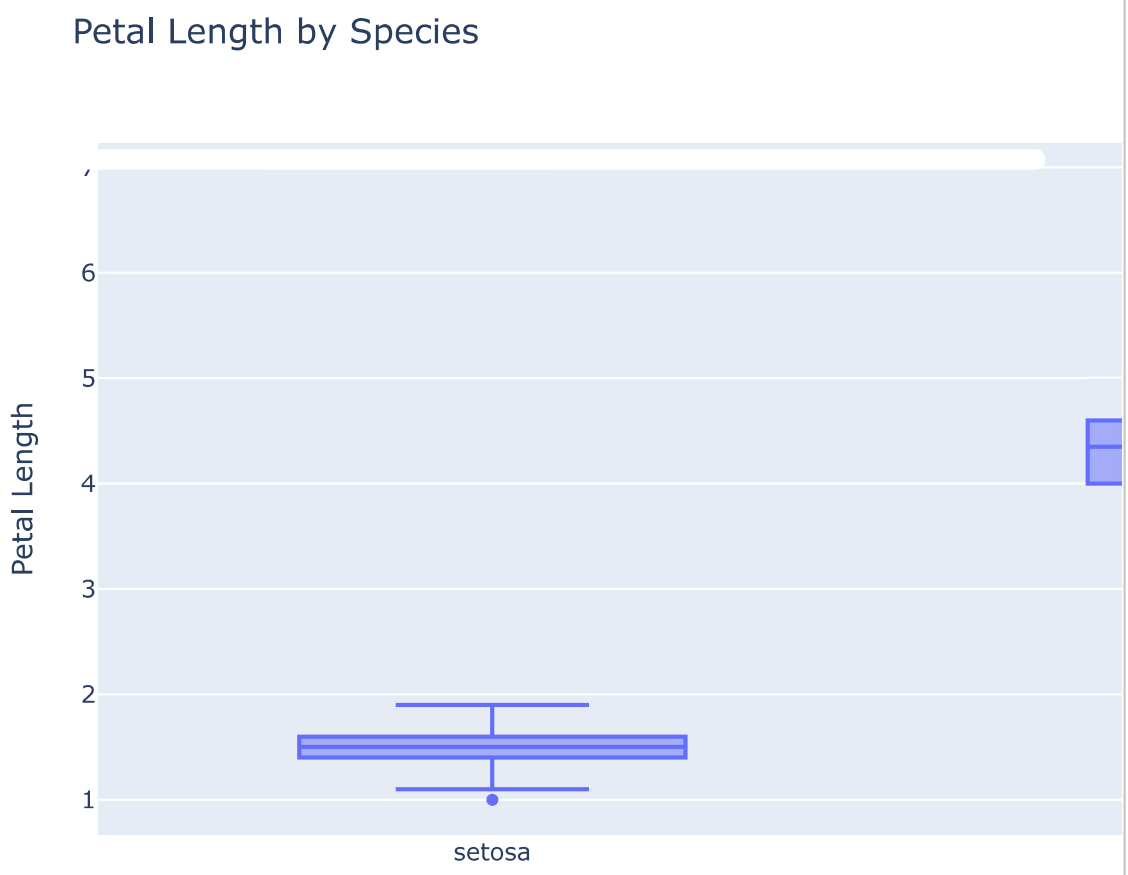


In this scatterplot made using Plotly, you can hover over individual data points to see their values, zoom in and out, and pan across the plot.

Create Box Plot of Petal Length by Species

```
# Create a box plot
fig = px.box(df, x="species", y="petal_length", title="Petal Length by Speci
             labels={'species': 'Species', 'petal_length': 'Petal Length'})

# Show the plot
```

```
fig.show()
```



In this boxplot made using Plotly, you can hover over each boxplot, and the given information will be displayed.

- maximum value
- minimum value
- first quartile (q1)
- third quartile (q3)

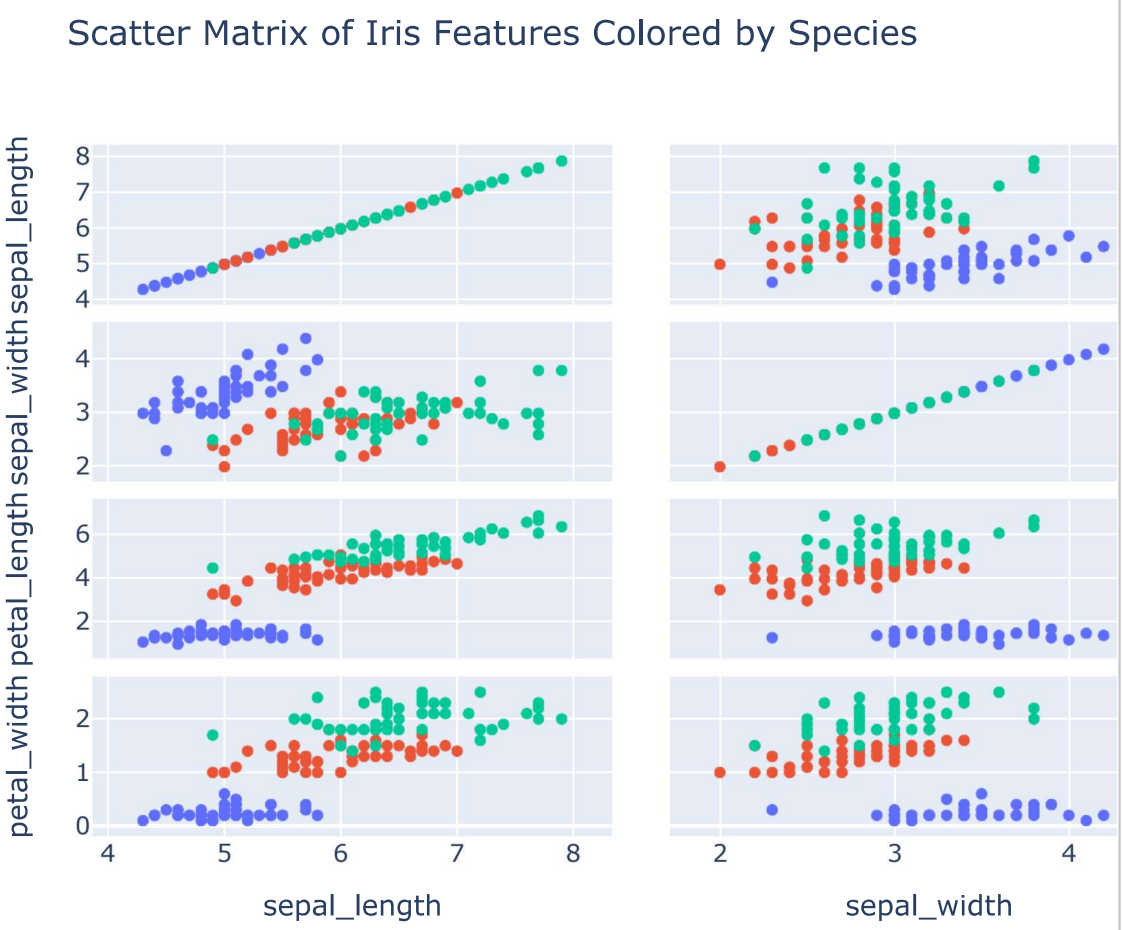
You can also zoom in to view individual plots.

Create Scatter Matrix of All Features Colored by Species

- In Plotly, a scatter matrix (also known as a pair plot or a scatterplot matrix) is a plot that allows you to visualize the pairwise relationships between multiple variables in a dataset.
- It is used to quickly identify correlations, patterns, and trends among different variables.

- Each cell in the scatter matrix shows a scatter plot of two variables, with one variable represented on the x-axis and the other on the y-axis.
- The diagonal cells often display histograms or density plots of the individual variables.

```
# Create a scatter matrix
fig = px.scatter_matrix(df, dimensions=["sepal_length", "sepal_width", "peta
                        color="species", title="Scatter Matrix of Iris Featu

# Show the plot
fig.show()
```



- The scatter matrix visualizes the relationships between the sepal length, sepal width, petal length, and petal width of iris flowers, with different species colored differently.
- This allows for a quick visual assessment of how these variables correlate with each other and how they are distributed across different species.

Create a Heatmap to Show Correlation of Features

- Heatmaps in Plotly are used for visualizing and analyzing data across two dimensions.
- They represent data using a grid of colored squares, where each square